LES SYSTÈMES D'EXPLOITATION MULTIPROCESSEURS

Synthèse simplifiée : Fondements, Noyau, Ordonnancement, Concurrence et Accélérateurs (GPU/IA)

Document de cours condensé avec références bibliographiques et exemples simplifiés

---

Introduction : L'impasse physique et le virage du multicœur

Pendant trois décennies, la vitesse des logiciels augmentait automatiquement avec l'arrivée des nouveaux processeurs, sans modifier le code. Cette période dorée du *"Free Lunch"* **[Sutter, 2005]** reposait sur la complémentarité entre la loi de Moore et l'échelle de Dennard.

1. L'âge d'or du monocœur

La **loi de Moore** (1965) observait que le nombre de transistors doublait environ tous les deux ans **[Moore, 1965]**. En parallèle, l'**échelle de Dennard** (1974) prouvait que la consommation électrique par unité de surface restait constante lorsque les transistors rapetissaient. Cela permettait d'augmenter la fréquence d'horloge sans faire chauffer exagérément la puce **[Dennard et al., 1974]**.

> **ANALOGIE : L'AMÉLIORATION DU MOTEUR UNIQUE**
> C'était comme améliorer une voiture sur une route à une voie : chaque année, le moteur tournait deux fois plus vite, réduisant la durée du trajet sans rien changer à la conduite.

2. La rupture physique : les trois murs (vers 2005)

Vers 2005, la miniaturisation s'est heurtée à des limites atomiques, provoquant l'effondrement de l'échelle de Dennard **[Silberschatz et al., 2018]** :

- **Mur de la puissance (Power Wall) :** Les fuites électriques font surchauffer le silicium.
- **Mur de la fréquence (Frequency Wall) :** Bloqué à 3-4 GHz sous peine de détruire le composant.
- **Mur de la mémoire (Memory Wall) :** La RAM est trop lente par rapport au processeur qui passe son temps à l'attendre.

3. Le paradoxe et la transition multicœur

Les besoins en calcul continuant d'exploser, les fondeurs ont placé **plusieurs cœurs de calcul** sur une même puce. Ne pouvant plus accélérer un cœur unique, on parallélise le travail. La charge bascule alors sur le système d'exploitation, qui doit coordonner ces cœurs sans conflit **[Tanenbaum & Bos, 2015]**.

> **ANALOGIE : LE CONVOI DE CAMIONS**
> Ne pouvant créer une voiture roulant à 1000 km/h, on envoie dix camions roulant à 100 km/h. La livraison globale augmente, mais il faut un régulateur de trafic efficace.

4. Définition

Un **système d'exploitation multiprocesseur** distribue les programmes sur plusieurs cœurs physiques ou logiques, gère la mémoire partagée et sécurise les accès simultanés **[Stallings, 2018]**.

> *S'exécutant lui-même sur ces processeurs, comment le noyau s'organise-t-il pour éviter d'être corrompu ?*

Partie 1 : Les modèles d'OS et l'organisation du noyau

1.1 Définition du noyau

Le **noyau (kernel)** est le cœur privilégié de l'OS qui gère la mémoire, les processus et les périphériques [Love, 2010].

1.2 Le modèle asymétrique (AMP)

Un processeur **maître** exécute le noyau, tandis que les processeurs **esclaves** exécutent les applications utilisateur [Silberschatz et al., 2018].

- *Avantage :* Simple à programmer, pas de conflit dans le noyau.
- *Inconvénient :* Le maître est un goulot d'étranglement et un point de défaillance unique (SPOF).

> **ANALOGIE : LA CUISINE À CHEF UNIQUE**
> Plusieurs commis préparent des plats, mais un seul chef détient les clés de la réserve. Les commis font la queue dès qu'ils ont besoin d'un ingrédient.

1.3 Le modèle symétrique (SMP)

Tous les processeurs sont égaux et peuvent exécuter le code du noyau [Tanenbaum & Bos, 2015]. C'est le choix des OS actuels (Linux, Windows, macOS).

- *Avantages :* Charge bien répartie, meilleure tolérance aux pannes.
- *Nouveau problème :* Deux cœurs risquent de modifier simultanément les données du noyau.

1.4 Tableau comparatif AMP / SMP

Critère	Modèle Asymétrique (AMP)	Modèle Symétrique (SMP)
Exécution noyau	Réservée au maître uniquement.	Partagée par tous les cœurs.
Goulot d'étranglement	Élevé (saturation du maître).	Faible (dépend des verrous).
Passage à l'échelle	Très limité.	Excellent (dizaines de cœurs).

1.5 L'organisation des verrous dans le noyau SMP

- **Verrou global (Big Kernel Lock - BKL) :** Un seul verrou pour tout le noyau. Simple mais retombe sur le goulot de l'AMP [Bovet & Cesati, 2005]. (Retiré dans Linux 2.6.39 en 2011).
- **Verrous fins :** Un verrou par structure de données. Excellent parallélisme, mais code complexe [Love, 2010].

> *Une fois le noyau protégé, comment répartit-on les processus utilisateur entre les cœurs ?*

Partie 2 : L'ordonnancement des processus

2.1 Structure des files d'attente

- **File globale unique :** Équilibrage parfait, mais tous les cœurs se disputent le verrou de la file **[Silberschatz et al., 2018]**.
- **Files locales par CPU :** Pas de conflit sur le verrou et meilleur usage du cache, mais risque de déséquilibre.

> **ANALOGIE : CAISSES DE SUPERMARCHÉ**
> File unique (banque) = équilibrage parfait mais bouchon à l'entrée. Files séparées (supermarché) = fluide, mais vous risquez d'être bloqué derrière un caddie plein.

2.2 Équilibrage de charge et Affinité

L'OS associe des files locales avec le **Work Stealing** (un cœur inactif "vole" du travail à un voisin surchargé) **[Tanenbaum & Bos, 2015]**. L'**affinité processeur** cherche à garder une tâche sur le même cœur pour réutiliser la mémoire cache rapide L1/L2.

2.3 Topologie NUMA

Sur les gros serveurs **NUMA (Non-Uniform Memory Access)**, la RAM est rattachée à des groupes de processeurs. Lire sa mémoire locale est très rapide, lire la mémoire distante d'un autre socket est lent. L'ordonnanceur regroupe le calcul et la mémoire sur le même nœud local **[Bovet & Cesati, 2005]**.

> *Pour éviter que ces calculs parallèles ne corrompent la mémoire, la synchronisation est indispensable.*

Partie 3 : Synchronisation, concurrence et pièges logiciels

3.1 Conditions de concurrence et instructions atomiques

Une **condition de concurrence (race condition)** survient lorsque le résultat dépend de l'ordre d'exécution de deux cœurs sur une même variable **[Stallings, 2018]**. Couper les interruptions sur un seul cœur ne suffit plus en SMP. On utilise des instructions matérielles **atomiques** (ex: `Compare-and-Swap`).

3.2 Les outils : Spinlock vs Mutex

- **Spinlock :** Le cœur boucle en attente active (*busy waiting*). Ultra-rapide mais brûle du CPU. Idéal pour de très courtes attentes **[Love, 2010]**.
- **Mutex :** Le processus s'endort, libérant le cœur. Économe, mais coûteux en changement de contexte.

3.3 Les pièges : Interblocage et Faux partage

- **Deadlock (Interblocage) :** Deux processus s'attendent mutuellement. (Conditions de Coffman **[Coffman et al., 1971]**).
- **Faux partage (False Sharing) :** Deux variables indépendantes situées sur la même ligne de cache de 64 octets provoquent des allers-retours inutiles entre cœurs, effondrant les performances **[Tanenbaum & Bos, 2015]**.

> *Pourquoi ne suffit-il pas d'ajouter des dizaines de cœurs pour accélérer un programme ?*

Partie 4 : Limites théoriques et évolution

4.1 La loi d'Amdahl

La **loi d'Amdahl** (1967) démontre que l'accélération maximale est bridée par la partie séquentielle (1 - p) qui ne peut pas être parallélisée (les verrous) **[Amdahl, 1967]** :

```
Accélération = 1 / [ (1 - p) + (p / n) ]
```

Avec 5 % de code séquentiel (1 - p = 0,05), l'accélération maximale absolue avec une infinité de cœurs est limitée à **20**.

4.2 L'ordonnancement asymétrique moderne (big.LITTLE)

Face au mur de la puissance, les puces modernes (ARM big.LITTLE, Apple Silicon, Intel) combinent des cœurs de Performance (P-cores) et d'Efficacité (E-cores) **[Silberschatz et al., 2018]**. L'OS oriente les tâches lourdes vers les P-cores et le travail de fond vers les E-cores.

> *Aujourd'hui, l'évolution dépasse le CPU : le système doit gérer des puces spécialisées comme les GPU et les puces d'IA.*

Partie 5 : L'ère des accélérateurs spécialisés (GPU et Puces IA)

5.1 Le GPU (Graphics Processing Unit)

Le GPU contient des centaines ou milliers de cœurs simples exécutant la même opération sur des volumes massifs de données (parallélisme SIMD/SIMT) [Hennessy & Patterson, 2017]. Initialement conçu pour les graphismes 3D, il est central pour le calcul scientifique et l'IA.

- **Rôle de l'OS :** Dialogue via un pilote (driver), gestion de la mémoire dédiée ou partagée, allocation sécurisée du GPU entre applications (navigateur, jeu, vidéo), et régulation d'énergie.
- *Nuance :* Sur PC de bureau, le GPU possède sa propre VRAM dédiée. Sur smartphone/SoC, le GPU et le CPU partagent la même mémoire RAM physique, modifiant la gestion des données par l'OS.

5.2 Les puces d'IA (NPU, TPU)

Les **NPU (Neural Processing Unit)** et **TPU (Tensor Processing Unit)** sont des circuits accélérateurs optimisés pour les calculs de réseaux de neurones (multiplications de matrices) [Hennessy & Patterson, 2017]. Ils sont très économes en énergie.

- **Exemples concrets :** Apple Neural Engine, Google Tensor sur Pixel, Qualcomm Hexagon NPU.
- **Usage typique :** Traitement d'images, reconnaissance vocale, traduction locale sans appel aux serveurs cloud.
- **Rôle de l'OS :** Répartir intelligemment la tâche entre CPU, GPU et NPU, et offrir des frameworks unifiés (ex: Core ML, Android NNAPI).

5.3 Le smartphone : Un SoC où tout cohabite

Le smartphone est l'exemple parfait du **SoC (System on Chip)** : une unique puce regroupe le CPU (big.LITTLE), le GPU, le NPU, le modem et le processeur d'image (ISP) [Tanenbaum & Bos, 2015]. L'OS mobile (Android, iOS) doit arbitrer la consommation d'énergie et la chauffe plutôt que la seule vitesse pure.

5.4 Ce qui change pour l'OS

- **Ordonnancement :** L'OS ordonnance des blocs de travail sur des unités hétérogènes. L'ordonnancement d'un GPU ou d'un NPU est géré par des grilles de calculs, pas par des tranches de temps classiques.
- **Mémoire & Énergie :** Réduire les copies de mémoire inutiles entre puces et éteindre les blocs inactifs (clock-gating / power-gating).
- **Firmware & Sécurité :** Un firmware autonome tourne souvent sur l'accélérateur. L'OS communique avec la puce via des interfaces d'appels et des pilotes.

Conclusion

L'évolution des systèmes d'exploitation est une suite de réponses aux contraintes physiques du silicium. De l'asymétrie initiale au SMP, du découpage des verrous à l'ordonnancement NUMA et hybride, l'OS s'est constamment adapté. Aujourd'hui, avec l'intégration du multicœur, de l'asymétrie matérielle et des puces spécialisées (GPU/NPU) au sein d'un même SoC, l'OS devient le chef d'orchestre d'une mosaïque de calculateurs hétérogènes.

Bibliographie et Références Académiques

1. **Amdahl, G. M. (1967).** *Validity of the single processor approach to achieving large scale computing capabilities.* AFIPS Conference Proceedings, 30, 483-485.
2. **Bovet, D. P., & Cesati, M. (2005).** *Understanding the Linux Kernel* (3rd ed.). O'Reilly Media.
3. **Coffman, E. G., Elphick, M., & Shoshani, A. (1971).** *System Deadlocks.* ACM Computing Surveys, 3(2), 67-78.
4. **Dennard, R. H., et al. (1974).** *Design of ion-implanted MOSFET's with very small physical dimensions.* IEEE Journal of Solid-State Circuits, 9(5), 256-268.
5. **Hennessy, J. L., & Patterson, D. A. (2017).** *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.
6. **Love, R. (2010).** *Linux Kernel Development* (3rd ed.). Addison-Wesley Professional.
7. **Moore, G. E. (1965).** *Cramming more components onto integrated circuits.* Electronics, 38(8), 114-117.
8. **Silberschatz, A., Galvin, P. B., & Gagne, G. (2018).** *Operating System Concepts* (10th ed.). Wiley.
9. **Stallings, W. (2018).** *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
10. **Sutter, H. (2005).** *The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software.* Dr. Dobb's Journal, 30(3), 16-22.
11. **Tanenbaum, A. S., & Bos, H. (2015).** *Modern Operating Systems* (4th ed.). Pearson.